VECTOR

MICROSAR OS HAL

Technical Reference

ARM Cortex-M

Version 1.01.00

Authors	Alexander Lerach, Bilal Parvez, Lukas Rohrwild
Status	Released

Document Information

History

Author	Date	Version	Remarks
Alexander Lerach, Bilal Parvez	2020-03-24	1.0.0	First released version.
Ahmed El Amri, Lukas Rohrwild	2020-04-30	1.1.0	Change family name from “IMX8” to “i.MX 8x” Added chapter Exception Context Manipulation.

Reference Documents

No.	Source	Title	Version
[1]	Vector	MICROSAR OS Technical Reference	See delivery information



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2020 Vector Informatik GmbH

Contents

1 Component History 8

2 Introduction 9
2.1 Hardware Overview 9

3 Functional Description 11
3.1 Unhandled Interrupts 11
3.2 Unhandled Exceptions 11
3.3 PSR Register Handling 11
3.4 Exception Context Manipulation 12

4 Integration 13
4.1 Hardware Software Interface 13
4.1.1 Generic Cortex-M 13
4.1.1.1 Context Registers 13
4.1.1.2 Core Registers 13
4.1.1.3 Core MPU Registers (Optional) 13
4.1.1.4 INTC Registers 14
4.1.2 ATSAMv7x Family 14
4.1.2.1 Context Registers 14
4.1.2.2 Core Registers 14
4.1.2.3 Core MPU Registers 15
4.1.2.4 INTC Registers 15
4.1.3 S32K14x Family 15
4.1.3.1 Context Registers 15
4.1.3.2 Core Registers 15
4.1.3.3 System MPU Registers 16
4.1.3.4 INTC Registers 16
4.1.3.5 Timer Module Registers 17
4.1.4 TDA2x 17
4.1.4.1 Context Registers 17
4.1.4.2 Core Registers 17
4.1.4.3 INTC Registers 17
4.1.4.4 IPU Registers 18
4.1.4.5 Spinlock Registers 18
4.1.5 Traveo II Family 18
4.1.5.1 Context Registers 18
4.1.5.2 Core Registers 18
4.1.5.3 Core MPU Registers 19

4.1.5.4 INTC Registers 19
4.1.5.5 IPC Registers 19
4.1.5.6 Timer Module Registers 20
4.1.6 i.MX 8x Family 20
4.1.6.1 Context Registers 20
4.1.6.2 Core Registers 20
4.1.6.3 Core MPU Registers 21
4.1.6.4 INTC Registers 21
4.1.6.5 TPM Registers 21
4.1.6.6 MU Registers 22
4.1.7 S32x Family 22
4.1.7.1 Context Registers 22
4.1.7.2 Core Registers 22
4.1.7.3 Core MPU Registers 22
4.1.7.4 INTC Registers 23
4.1.7.5 Spinlock 23
4.1.7.6 Mode Entry Module 23
4.1.7.7 Reset Generation Module 23
4.1.7.8 STM Registers 23
4.1.8 S32K2TV/S32K3xx Family 24
4.1.8.1 Context Registers 24
4.1.8.2 Core Registers 24
4.1.8.3 Core MPU Registers 24
4.1.8.4 INTC Registers 25
4.2 Configuration of X-Signal 25
4.2.1 X-Signal ISR Interrupt Sources 25
4.3 Configuration of Interrupt Mapping 25
4.3.1 Traveo II 25
4.3.2 i.MX 8x 26

5 Platform Restrictions 27
5.1 Exception Handlers 27
5.2 Memory 27
5.3 Stack usage measurement 28
5.4 Section Symbols 28
5.5 Floating Point Context 28

6 Derivative Restrictions 29
6.1.1 General 29
6.1.2 S32K Family 29
6.1.3 S32x Family 30

6.1.4 TDA2x Family 30
6.1.5 i.MX 8x Family 30

7 Contact 31

Tables

Table 1-1 Component history 8
Table 2-1 Supported ARM Hardware 9
Table 2-2 Supported ARM compilers 10

© 2020 Vector Informatik GmbH

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features

Table 1-1 Component history

2 Introduction

This document describes platform specific details about the usage and functions of “MICROSAR OS” on the ARM Cortex-M derivative family. General information about “MICROSAR OS” can be found in [1].

2.1 Hardware Overview

The following table summarizes information about MICROSAR OS. It gives detailed information about the derivatives and supported compilers of the ARM Cortex-M derivative family. As very important information the documentations of the hardware manufacturers are listed. MICROSAR OS is based upon these documents in the given version.

Derivative Family	Hardware Manufacturer Document Name	Document Version
i.MX8 x	i.MX 8DualXPlus/8QuadXPlus Application Processor Reference Manual, i.MX 8QuadMax Applications Processor Reference Manual	Rev. B, 5/2018, Rev. F, 10/2019
ATSAMV7x	SAM v70 Datasheet	Rev.11297D, 06/2016
S32K14x	NXP/Freescale - S32K14x Series Reference Manual - Supports S32K142, S32K144, S32K146, and S32K148	Rev. 3, 03/2017
Generic_CortexM	ARMv7-M Architecture Reference Manual	v.E.b 12/2015
Generic_CortexM0	ARMv6-M Architecture Reference Manual	ARM DDI 0419D 19 May 2017
TDA2x	TDA2x Technical Reference Manual	SPRUI29D, November 2015
CYT2Bx	Traveo II Automotive Body Controller Entry Family Architecture Technical Reference Manual	Rev.*B, February 2018
CYT4Bx	Traveo II Automotive Body Controller High Family Architecture Technical Reference Manual	Rev. *A, September 2018
S32x	S32V344 Reference Manual S32G Reference Manual	Rev.1 Draft M, Juni 2018 S32G274ARM, Rev.1, Nov 2019
NXP S32K2TV/ S32K3xx	S32K2xx Reference Manual	S32K2xxRM Rev.1 Draft N, 07/2019

Table 2-1 Supported ARM Hardware

Green Hills (GHS)	2013.5.4 and later
IAR	7.70.x and later
TI	15.12.3.LTS
ARM CC	5.06u1 and later
GCC Linaro Distribution	gcc-linaro-7.1.1-2017.08-i686-mingw32_arm-eabi
Tasking	v6.2r2p2

Table 2-2 Supported ARM compilers

3 Functional Description

3.1 Unhandled Interrupts

In case of an unhandled interrupt request the application can obtain the source number of the triggering interrupt request by the OS API Os_GetUnhandledIrq() (see [1] for details). On the ARM Cortex-M derivative family, the 32-bit value returned by the API represents the source of the interrupt as found in the exception table, if it is a Core local interrupt without mapping. In case the interrupt controller supports mapping and the interrupt is mapped, the system interrupt source is provided.

3.2 Unhandled Exceptions

In case of an unhandled exception the application can obtain the source number of the triggering exception by the OS API Os_GetUnhandledExc() (see [1] for details). On the ARM Cortex-M derivative family, the 32-bit value returned by the API represents the exception source number. The unhandled exception numbers range from 2 to 14.

Exception Number	Comment
2	NMI (Non Maskable Interrupt) exception is per default unhandled. Can be triggered through hardware or software.
3	Hard Fault is handled by the OS when memory protection is enabled is unhandled otherwise.
4	MemManage is handled by the OS when memory protection is enabled is unhandled otherwise.
5	BusFault is handled by the OS when memory protection is enabled is unhandled otherwise.
6	UsageFault is triggered through non-memory related faults.
7-10	Are reserved on the platform and should not happen.
12	DebugMonitor is unhandled.
13	Is reserved on the platform and should not happen.
14	PendSV is not utilized by the OS.

3.3 PSR Register Handling

PSR.T bit handling	The Thumb execution state bit is left unchanged during runtime. This bit is controlled by hardware.
PSR.N/Z/C/V handling	The condition code flags bits are left unchanged during runtime. Thes bits are controlled by hardware.

3.4 Exception Context Manipulation

The Cortex-M derivative family does not support the feature to read and modify the interrupted context in case of a hardware exception by using the OS APIs Os_GetExceptionContext and Os_SetExceptionContext (see [1] for details).

4 Integration

4.1 Hardware Software Interface

The following chapter describes the Hardware-Software Interface of the ARM Cortex-M derivative family.

4.1.1 Generic Cortex-M

4.1.1.1 Context Registers

- R4-R11
- PC
- LR
- SP
- PSR
- CONTROL
- BASEPRI

4.1.1.2 Core Registers

- MPIDR
- PRIMASK
- FAULTMASK
- CCR
- SHPR1
- SHPR2
- SHPR3
- SHCSR
- SYST_CSR
- SYST_RVR
- SYST_CVR
- SYST_CALIB

4.1.1.3 Core MPU Registers (Optional)

- MPU_TYPE
- MPU_CTRL
- MPU_RNR
- MPU_RBAR
- MPU_RASR

4.1.1.4 INTC Registers

- SHPR
- NVIC_ISER
- NVIC_ICER
- NVIC_ISPR
- NVIC_ICPR
- NVIC_IABR
- NVIC_IPR
- ICSR
- VTOR
- STIR

4.1.2 ATSAMv7x Family

4.1.2.1 Context Registers

- R4-R11
- PC
- LR
- SP
- PSR
- CONTROL
- BASEPRI

4.1.2.2 Core Registers

- MPIDR
- PRIMASK
- FAULTMASK
- CCR
- SHPR1
- SHPR2
- SHPR3
- SHCSR
- SYST_CSR
- SYST_RVR

- SYST_CVR
- SYST_CALIB

4.1.2.3 Core MPU Registers

- MPU_TYPE
- MPU_CTRL
- MPU_RNR
- MPU_RBAR
- MPU_RASR

4.1.2.4 INTC Registers

- SHPR
- NVIC_ISER
- NVIC_ICER
- NVIC_ISPR
- NVIC_ICPR
- NVIC_IABR
- NVIC_IPR
- ICSR
- VTOR
- STIR

4.1.3 S32K14x Family

4.1.3.1 Context Registers

- R4-R11
- PC
- LR
- SP
- PSR
- CONTROL
- BASEPRI

4.1.3.2 Core Registers

- MPIDR
- PRIMASK

- FAULTMASK
- CCR
- SHPR1
- SHPR2
- SHPR3
- SHCSR
- SYST_CSR
- SYST_RVR
- SYST_CVR
- SYST_CALIB

4.1.3.3 System MPU Registers

- SMPU_CESR
- SMPU_RGDn_WORD0
- SMPU_RGDn_WORD1
- SMPU_RGDn_WORD2
- SMPU_RGDn_WORD3
- SMPU_RGDAAC0

4.1.3.4 INTC Registers

- SHPR
- NVIC_ISER
- NVIC_ICER
- NVIC_ISPR
- NVIC_ICPR
- NVIC_IABR
- NVIC_IPR
- ICSR
- VTOR
- STIR

4.1.3.5 Timer Module Registers

- FTM_SC
- FTM_CNT
- FTM_CnSC
- FTM_CnV

4.1.4 TDA2x

4.1.4.1 Context Registers

- R4-R11
- PC
- LR
- SP
- PSR
- CONTROL
- BASEPRI

4.1.4.2 Core Registers

- MPIDR
- PRIMASK
- FAULTMASK
- CCR
- SHPR1
- SHPR2
- SHPR3
- SHCSR
- SYST_CSR
- SYST_RVR
- SYST_CVR
- SYST_CALIB

4.1.4.3 INTC Registers

- SHPR
- NVIC_ISER

- NVIC_ICER
- NVIC_ISPR
- NVIC_ICPR
- NVIC_IABR
- NVIC_IPR
- ICSR
- VTOR
- STIR

4.1.4.4 IPU Registers

- CORTEXM4_CTRL_REG
- CORTEXM4_RW_PID1

4.1.4.5 Spinlock Registers

- SPINLOCK_SYSCONFIG
- SPINLOCK_SYSTATUS
- SPINLOCK_LOCK_REG_0

4.1.5 Traveo II Family

4.1.5.1 Context Registers

- R4-R11
- PC
- LR
- SP
- PSR
- CONTROL
- BASEPRI

4.1.5.2 Core Registers

- MPIDR
- PRIMASK
- FAULTMASK
- CCR
- SHPR1
- SHPR2

- SHPR3
- SHCSR
- SYST_CSR
- SYST_RVR
- SYST_CVR
- SYST_CALIB
- CPUSS_IDENTITY

4.1.5.3 Core MPU Registers

- MPU_TYPE
- MPU_CTRL
- MPU_RNR
- MPU_RBAR
- MPU_RASR

4.1.5.4 INTC Registers

- SHPR
- NVIC_ISER
- NVIC_ICER
- NVIC_ISPR
- NVIC_ICPR
- NVIC_IABR
- NVIC_IPR
- ICSR
- VTOR
- STIR
- CPUSS_CM4_SYSTEM_INT_CTL (Body Entry)
- CPUSS_CM4_INTm_STATUS (Body Entry, m = 0…7)
- CPUSS_CM7_0_SYSTEM_INT_CTL (Body High)
- CPUSS_CM7_1_SYSTEM_INT_CTL (Body High)
- CPUSS_CM7_0_INTm_STATUS (Body High, m = 0…7)
- CPUSS_CM7_1_INTm_STATUS (Body High, m = 0…7)

4.1.5.5 IPC Registers

- IPC_STRUCT7_ACQUIRE (Body High, reserved for Spinlocks)

- IPC_STRUCT7_RELEASE (Body High, reserved for Spinlocks)
- IPC_STRUCTm_NOTIFY (Body High, m = 0…6)
- IPC_STRUCTm_INTR (Body High, m = 0…6)
- IPC_STRUCTm_INTR_MASK (Body High, m = 0…6)

4.1.5.6 Timer Module Registers

- TCPWM0_GRP2_CNTm_CTRL (Body Entry, reserved for HRT, m= 0…4)
- TCPWM0_GRP2_CNTm_COUNTER (Body Entry, reserved for HRT, m= 0…4)
- TCPWM0_GRP2_CNTm_CC0 (Body Entry, reserved for HRT, m= 0…4)
- TCPWM0_GRP2_CNTm_TR_CMD (Body Entry, reserved for HRT, m= 0…4)
- TCPWM0_GRP2_CNTm_INTR (Body Entry, reserved for HRT, m= 0…4)
- TCPWM0_GRP2_CNTm_INTR_MASK (Body Entry, reserved for HRT, m= 0…4)

4.1.6 i.MX 8x Family

4.1.6.1 Context Registers

- R4-R11
- PC
- LR
- SP
- PSR
- CONTROL
- BASEPRI

4.1.6.2 Core Registers

- MPIDR
- PRIMASK
- FAULTMASK
- CCR
- SHPR1
- SHPR2
- SHPR3
- SHCSR
- SYST_CSR
- SYST_RVR

- SYST_CVR
- SYST_CALIB

4.1.6.3 Core MPU Registers

- MPU_TYPE
- MPU_CTRL
- MPU_RNR
- MPU_RBAR
- MPU_RASR

4.1.6.4 INTC Registers

- SHPR
- NVIC_ISER
- NVIC_ICER
- NVIC_ISPR
- NVIC_ICPR
- NVIC_IABR
- NVIC_IPR
- ICSR
- VTOR
- STIR
- IRQSTEER_CHAN0CTL
- IRQSTEER_CHAN0_MINTDIS
- IRQSTEER_(CHAN0_MASK0 – CHAN0_MASK15)
- IRQSTEER_(CHAN0_STATUS0 – CHAN0_STATUS15)

4.1.6.5 TPM Registers

- TPM_SC
- TPM_CONF
- TPM_QDCTRL
- TPM_C0SC
- TPM_C0V
- TPM_CNT
- TPM_MOD

4.1.6.6 MU Registers

- MUA_ACR (MU0_A0, Reserved for Xsignals)
- MUB_BCR (MU0_B, Reserved for Xsignals)
- MUB_BSR (MU0_B, Reserved for Xsignals)

4.1.7 S32x Family

4.1.7.1 Context Registers

- R4-R11
- PC
- LR
- SP
- PSR
- CONTROL
- BASEPRI

4.1.7.2 Core Registers

- MPIDR
- PRIMASK
- FAULTMASK
- CCR
- SHPR1
- SHPR2
- SHPR3
- SHCSR
- SYST_CSR
- SYST_RVR
- SYST_CVR
- SYST_CALIB

4.1.7.3 Core MPU Registers

- MPU_TYPE
- MPU_CTRL
- MPU_RNR
- MPU_RBAR

- MPU_RASR

4.1.7.4 INTC Registers

- SHPR
- NVIC_ISER
- NVIC_ICER
- NVIC_ISPR
- NVIC_ICPR
- NVIC_IABR
- NVIC_IPR
- ICSR
- VTOR
- STIR
- MSCM_IRSPRC
- MSCM_IRCPISR
- MSCM_IRCPIGR
- MSCM_CPUXNUM

4.1.7.5 Spinlock

- XRDC_ HWCFG1
- SEMA42_GATE0
- SEMA42_RSTGT

4.1.7.6 Mode Entry Module

- MC_ME_CTL_KEY
- MC_ME_PRTN0_COREn_ADDR , n = [1,2]
- MC_ME_PRTN0_COREn_PCONF , n = [1,2]
- MC_ME_PRTN0_COREn_PUPD , n = [1,2]
- MC_ME_PRTN0_COREn_STAT , n = [1,2]

4.1.7.7 Reset Generation Module

- RGM_PRST
- RGM_PSTAT

4.1.7.8 STM Registers

- STM_TS_CR
- STM_TS_CNT

- STM_TS_CCR0
- STM_TS_CIR0
- STM_TS_CMP0

4.1.8 S32K2TV/S32K3xx Family

4.1.8.1 Context Registers

- R4-R11
- PC
- LR
- SP
- PSR
- CONTROL
- BASEPRI

4.1.8.2 Core Registers

- MPIDR
- PRIMASK
- FAULTMASK
- CCR
- SHPR1
- SHPR2
- SHPR3
- SHCSR
- SYST_CSR
- SYST_RVR
- SYST_CVR
- SYST_CALIB

4.1.8.3 Core MPU Registers

- MPU_TYPE
- MPU_CTRL
- MPU_RNR
- MPU_RBAR
- MPU_RASR

4.1.8.4 INTC Registers

- SHPR
- NVIC_ISER
- NVIC_ICER
- NVIC_ISPR
- NVIC_ICPR
- NVIC_IABR
- NVIC_IPR
- ICSR
- VTOR
- STIR
- MSCM_IRSPRC
- MSCM_IRCPISR
- MSCM_IRCPIGR
- MSCM_CPUXNUM

4.2 Configuration of X-Signal

Logical Priority	A low number for OsIsrInterruptPriority attribute means a high logical priority
X-Signal ISR Interrupt Priority	Aside from the priority rules for the X-Signal receiver ISR described in [1] the OsIsrInterruptPriority can be chosen freely.

4.2.1 X-Signal ISR Interrupt Sources

TDA2x	Interrupt source 19 has to be used for the X-Signal ISRs.
S32x	Interrupt source 17 has to be used for the X-Signal ISRs.
CYT4Bx	Interrupt sources 0…6 have to be used for the X-Signal ISR. (IPC IRQs)
i.MX 8x	Interrupt source 45 has to be used for the X-Signal ISRs. (MU0_B0 IRQ)

4.3 Configuration of Interrupt Mapping

4.3.1 Traveo II

Supported Interrupt Type	CoreExternalIRQ0..CoreExternalIRQ7
ISR Category	Any
Supported on derivative families	Traveo II

The Interrupt Mapping feature on Traveo II is used to map Core external peripheral interrupt sources to one of the 8 CoreExternalIRQ channels.

After mapping an interrupt to any CoreExternalIRQ channel, the interrupt is added to the generated System Interrupt Vector Table. An interrupt not mapped to any type will be treated as a normal core local interrupt source.

4.3.2 i.MX 8x

Supported Interrupt Type	CoreExternalIRQ0..CoreExternalIRQ7
ISR Category	Any
Supported on derivative families	i.MX 8x

The Interrupt Mapping feature on i.MX 8x is used to map Core external peripheral interrupt sources to one of the 8 CoreExternalIRQ channels.There are limitations on which interrupts can be mapped to which sources.

After mapping an interrupt to any CoreExternalIRQ channel, the interrupt is added to the generated System Interrupt Vector Table. An interrupt not mapped to any type will be treated as a normal core local interrupt source.

5 Platform Restrictions

5.1 Exception Handlers

- The exception handler for Supervisor Call is implemented by the OS
- The exception handler for Systick is implemented by the OS

5.2 Memory



Caution
The MPU configuration must not contain the regions with the number higher than the number of available MPU regions minus 2. One region with the highest number is always reserved for the stack protection.

E.g. if 16 regions are available, only the region numbers from 0 to 14 (inclusive) are allowed.



Caution with ARMv7 MPU (PMSAv7)
Due to ARM MPU hardware restriction the sizes of ARM MPU regions and stack sizes must be configured with power of 2 values. An ARM CPU may be equiped with a non-ARM MPU, in that case the aforementioned limitation does not generally hold and the documentation of the MPU should be consulted.

5.3 Stack usage measurement



Caution with GCC compiler
If the feature “Stack Usage Measurement” is activated and one of the OS stacks (managed by the OS) is applied before calling Os_Init, then the optimization option tree-loop-distribute-patterns needs to be disabled.

5.4 Section Symbols



Note
When using the ARM compiler, the OS generator will not generate section start and stop symbols. However, the ARM linker will provide region-related symbols with special patterns (e.g. Image$$region_name$$Base or Load$$region_name$$Base), which can be used to configure start and end addresses of MPU regions, peripheral regions or access check region objects. Detailed information about the region-related symbols can be found in the user guide of the ARM compiler.

5.5 Floating Point Context



Note
When setting OsFpuUsage to ALL or INDIVIDUAL additional memory and runtime is required for FPU context handling. This affects the following derivative families:

> ARM Cortex-M



Caution
MICROSAR OS expects that the Lazy Context Save feature is disabled at startup (FPCCR.LSPEN bit is set to 0).

6 Derivative Restrictions

6.1.1 General



Caution with Interrupt Priorities
Due to ARM v7M architecture restriction on how to handle interrupts, the interrupt priority of any Category 2 interrupt (including CounterIsr) that can cause scheduling should be lower than all other Category 2 interrupts. This is needed to avoid unexpected delays in interrupt excecution and interrupt overruns.



Caution with exception vector table address
To avoid memory violations directly after boot phase, the exception vector table has to be linked correctly (derivative specific) by the user linker script.
e.g. ATSAMv7 Derivative expects that section OS_INTVEC_CORE<Core Id>_CODE is linked to address 0x00400000 (first available internal flash address).



Note for GCC compiler
The interrupt vector tables are in the sections with the name “<Core_Name>_VectorTable_Section”. These sections need to be 128 bytes aligned.

6.1.2 S32K Family



Caution
Region 0 of the System MPU is reserved for debugging functionality and could not be written by the core. This region is not available for user configuration.



Caution
Due to FTM module hardware restriction (only 16 Bit count and compare registers), the prescaler of the timer module is set to 32 by software. This is needed to achieve an adequate timer resolution. This needs to be considered when configuring the parameter SecondsPerTick for an HRT OS-Counter.

6.1.3 S32x Family



Caution
MICROSAR OS expects that Hardware Configuration 1 register (XRDC_HWCFG1) provides the correct domain number of the requesting bus master (Processor cores).

6.1.4 TDA2x Family



Caution
MICROSAR OS expects that register CORTEXM4_RW_PID1 is initialized with the physical core id before starting.

6.1.5 i.MX 8x Family



Caution
MICROSAR OS expects that on Multicore derivatives the Tightly Coupled Memory is enabled and the start of TCM (address 0x20000000) is initialized with the physical core id before starting.

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com